

HOT TOPICS · PATCHING AT SPEED

# Close the gap.

October 2026 brought a record Patch Tuesday, an emergency PHP release and an Oracle flaw probed within a week. Here is how to patch fast, safely and every time.

INSIDE

# Contents

A practical guide to patching at the speed attackers now move: know what you run, prioritise what matters, patch fast without breaking things, and make it a habit you can measure.

Foreword 03

Introduction 04

01 Know what you run 06

02 Patch at speed 11

03 Make it a habit 17

FOREWORD

# Patching used to be a monthly chore. It is now a race.

In October 2026 Microsoft released more than 1,400 security fixes in a single month, one of them for a flaw already under attack. Eight days later the PHP project shipped emergency fixes for two critical flaws in the language behind a large share of the web, and Amazon reported an exploitation attempt four days before they were made public. Within a week of Oracle's quarterly update, decoy servers were recording attempts against a new E-Business Suite flaw.

The pattern is clear. The time between a fix being released and attackers using the flaw has collapsed from weeks to days, and sometimes it is negative. AI is helping researchers find more flaws, which means more fixes, and it is making it cheaper to turn a fix into an exploit.

Most organisations do not fall behind because they do not care. They fall behind because their process was built for a slower world: monthly cycles, long test windows and no clear view of what they run. This playbook shows how to change that, step by step, with owners, checks and measures you can report to the board.

Our companion report, The Patch Gap, sets out the evidence in detail, with every figure sourced.




**The Transputec team**
Transputec

INTRODUCTION

# Fast, focused, and every time.

You cannot patch everything at once, and you do not need to. You need to know what you run, fix the few flaws that matter within days, and keep everything else on a steady, tested rhythm.



### Know what you run

A live inventory of systems, software and versions, with an owner for each. You cannot patch what you cannot see.



### Fix what matters first

Exploited and internet-facing flaws go to the front of the queue. Severity scores come third.



### Make it a habit

Clear targets, a pre-approved emergency lane and a monthly measure of how long fixes really take.

# The patch and the attack now arrive in the same week.

For exploited, internet-facing flaws, patching is incident response.
It is measured in hours and days, not maintenance windows.

01

PART 01

# Know what you run.

Speed starts with sight. Inventory, prioritisation and clear targets come before any patch is deployed.

01 Why the inventory comes first

02 How to prioritise

03 Patch targets by risk

KNOW WHAT YOU RUN

# You cannot patch what you cannot see

> **When PHP2Shell broke, the first question was not how to patch. It was where PHP was running.**

When PHP2Shell was announced on 21 October, the first instruction from both the NCSC and CISA was the same: find every PHP installation. Not patch, not mitigate. Find. Many organisations discovered they could not answer the question quickly, because PHP was running inside appliances, vendor products and forgotten web servers as well as their own sites.

That is the heart of the patch gap. Most organisations know their Windows laptops well, because a management tool reports on them every day. They know much less about web servers, appliances, cloud workloads, the software a supplier installed years ago, and the components buried inside other products.

A useful inventory answers four questions for every system: what it is, what software and versions it runs, whether it can be reached from the internet, and who owns it. The last question matters as much as the first. A system with no owner is a system nobody will patch at 9pm on a Wednesday.

You do not need a perfect inventory to start. Pull together what your device management, vulnerability scanner, cloud console and firewall already know. Add an external scan of your own internet-facing estate, because that is the view attackers have. Then fill the gaps system by system.

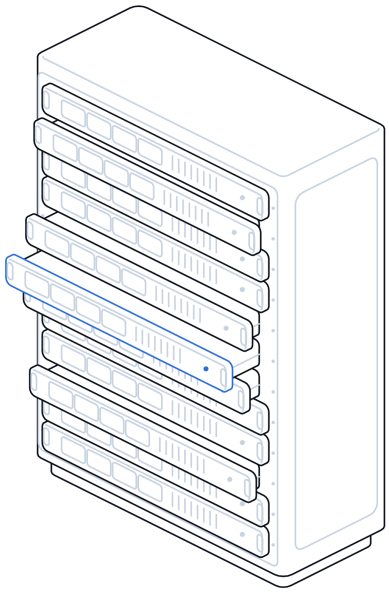
Keep it live. An inventory built for an audit and then left alone is out of date within weeks. Connect it to the tools that already see change, and review exceptions monthly.

Finally, include suppliers. Ask your managed service provider and key software vendors to tell you which of their products contain common components such as PHP, Java or OpenSSL, and to notify you when those components need urgent fixes.

INVENTORY

# Four sources that build a working inventory

Most of the data already exists. Bring it together and keep it current.



**01** **Device and endpoint management**

Management tools know laptops and many servers, with versions and update status.

**02** **Vulnerability scanning**

Authenticated scans find software versions, including components hidden in other products.

**03** **External attack surface**

Scan your public IP ranges and domains. It shows what attackers see and finds forgotten systems.

**04** **Cloud and supplier records**

Cloud consoles list workloads. Suppliers and your MSP list what they manage for you.

PRIORITISE

# What to fix first

Two questions decide urgency: is the flaw being exploited, and can an attacker reach it? A high severity score alone is not enough.

↑ KNOWN EXPLOITATION

FIX WITHIN DAYS

### Exploited, internal

Exploited flaws that need a click or internal access, like October's Windows Shell bypass. Patch fast.

ACT NOW

### Exploited and exposed

Emergency lane. Patch or mitigate within hours to days, isolate if needed and check for compromise.

NORMAL CYCLE

### Not exploited, internal

Routine updates in tested rings. Track them so they never slip past 14 days.

FIX FAST

### Not exploited, exposed

Critical flaws on internet-facing systems, such as Oracle EBS before probing began. Treat as near-urgent.

EXPOSURE TO THE INTERNET →

PATCH TARGETS

# Patch targets by risk tier

A starting point to agree with your board and suppliers. Targets run from the vendor's release, not from when you noticed.

| TIER | EXAMPLE | TARGET | IF YOU CANNOT MEET IT |
|---|---|---|---|
| **Exploited and internet-facing** | PHP2Shell on a public web server. | Hours to 3 days. | Mitigate, isolate or take offline. Check for compromise. |
| **Exploited, internal** | Windows Shell bypass on staff laptops. | Within 7 days, high-risk users first. | Extra email filtering and monitoring until done. |
| **Critical or high, exposed** | Oracle EBS flaw before attacks began. | Within 7 days. | Restrict access to trusted networks meanwhile. |
| **Critical or high, internal** | Most monthly operating system fixes. | Within 14 days, the Cyber Essentials limit. | Record an exception with an owner and end date. |
| **Medium and low** | Routine application updates. | Next monthly cycle. | Review at the monthly patch meeting. |

02

PART 02

# Patch at speed.

A fast lane for urgent fixes, testing that does not stall, and an honest plan for what cannot be patched.

01 The emergency process

02 Testing without delay

03 Legacy systems and compensating controls

EMERGENCY PROCESS

# The emergency lane, from alert to assurance

Agree this in advance, so nobody needs to convene a meeting to get permission when a critical flaw lands.

### Detect

Watch CISA's exploited list, NCSC alerts and vendor advisories daily. Your MSP should alert you within hours.

### Scope

Use the inventory to find every affected system within hours, internet-facing first.

### Contain

If you cannot patch at once, apply vendor mitigations, block exposure or take the system offline.

### Patch

Deploy to a small test group, then everywhere. Change approval is pre-agreed for this lane.

### Hunt

Check for signs of compromise, looking back to the earliest known attack date.

### Confirm

Verify every system is fixed, record the time taken and close the exceptions.

> **Pre-approval is the key. The emergency lane is a standard change, not a special one.**

TESTING

# Testing that protects you without slowing you down

Testing exists to catch breakage. It should never become the reason an exploited flaw stays open for weeks.

## 01 Patch in rings

Deploy to IT staff and a pilot group first, then a wider group, then everyone. Each ring takes hours for urgent fixes, days for routine ones.

## 02 Keep a test copy of key systems

A staging copy of business-critical applications lets you test vendor fixes the day they arrive.

## 03 Automate the basics

Scripted smoke tests confirm that core services start and key pages load, so you are not waiting on manual checks.

## 04 Have a rollback ready

Snapshots, backups and documented rollback steps make it safe to move fast.

## 05 Use rebootless updates where offered

Microsoft offers hotpatching for supported Windows 11 devices and some servers, cutting disruption.

REMEMBER

For an exploited, internet-facing flaw, the risk of not patching is almost always bigger than the risk of a failed update.

IN PRACTICE

# Three October flaws, three responses



ENDPOINTS

## A Windows zero-day in a record release

THE SITUATION

Over 1,400 fixes, one Windows Shell flaw already exploited and needing a click.

WHAT GOOD LOOKS LIKE

Exploited fix done in days; the rest follows the normal tested rings.

THE RESPONSE

Pull the exploited fix forward and push it to high-risk users within days.

WATCH OUT

Do not let the size of the release delay the one fix that matters.



WEB SERVERS

## PHP2Shell on a forgotten server

THE SITUATION

Critical PHP flaws, attempts seen before disclosure, old versions with no fix.

WHAT GOOD LOOKS LIKE

All exposed PHP fixed or offline within days, with logs checked for compromise.

THE RESPONSE

Find every PHP install, patch exposed ones first, isolate what cannot be patched.

WATCH OUT

PHP inside appliances and vendor products. Ask suppliers directly.



BUSINESS SYSTEMS

## An Oracle E-Business Suite flaw

THE SITUATION

Unauthenticated SQL injection, fixed in October, probed within a week.

WHAT GOOD LOOKS LIKE

Fix applied before attacks spread, with database activity monitored.

THE RESPONSE

Patch test, live and copies. Restrict the web tier to trusted networks meanwhile.

WATCH OUT

Disaster recovery and test copies are often missed.

# 04



# Legacy and end-of-life systems

Software that no longer gets security fixes is where the gap never closes. PHP 8.1 and older got no official fix for PHP2Shell.

### THE PROBLEM

Old systems often run critical processes, have no vendor support and cannot simply be switched off.

### FIND THEM

Flag every unsupported operating system, runtime and application in the inventory, with its owner.

### DECIDE

For each one: upgrade, replace, isolate or accept the risk formally, with a date and an executive sign-off.

### ISOLATE

Remove internet access, restrict who can reach it and monitor it closely. Cyber Essentials requires this for unsupported software.



### OWNER

**IT lead, with business owners of each legacy system**

Bring this to the monthly review, with a date for each action.

COMPENSATING CONTROLS

# When you cannot patch today

Mitigations buy time. They never replace the fix, and every one should have an end date.

### Take it offline

The safest option for an exposed system with an exploited flaw and no fix. UK public bodies did this for PHP2Shell.

### Block the attack path

Web application firewall rules, blocked endpoints or disabled features. Cloudflare called its PHP2Shell rules a stopgap.

### Reduce exposure

Restrict access to trusted networks or a VPN, and remove internet access where it is not needed.

### Watch closely

Raise logging and alerting on the system, and look for unusual processes, files and outbound traffic.

> Record every mitigation as an exception with an owner and a date to remove it.

03

PART 03

# Make it a habit.

Speed that lasts comes from measures, routines and partners that hold each other to account.

01 Measuring patch latency

02 This week, month and quarter

03 Working with your MSP

MEASURE

# Measuring patch latency

A handful of measures, reported monthly, tells you whether the gap is closing.

| MEASURE | WHAT IT TELLS YOU | GOOD LOOKS LIKE |
|---|---|---|
| **Time to patch exploited flaws** | How fast you close what attackers are using. | Days, measured from vendor release. |
| **Time to patch critical and high** | Whether you meet Cyber Essentials and your own targets. | All within 14 days, most within 7. |
| **Coverage** | Share of known systems reporting patch status. | Close to 100%, with gaps named. |
| **Exposed unpatched flaws** | What attackers can reach today. | Zero exploited; few critical, all with exceptions. |
| **Legacy count** | Unsupported systems still in use. | Falling every quarter, each with a plan. |

PLAN

# What to do this week, this month, this quarter

A practical sequence that builds speed without a big programme.

| WHEN | ACTION | OWNER | CHECK |
|---|---|---|---|
| **This week** | Confirm PHP2Shell, the Windows zero-day and Oracle EBS are fixed or mitigated. | IT lead or MSP | Evidence per system. |
| **This week** | Subscribe to CISA's exploited list and NCSC alerts. | Security lead | Alerts reaching a named inbox. |
| **This month** | Build a first inventory and external scan of exposed systems. | IT lead | Each system has an owner. |
| **This month** | Agree patch targets and a pre-approved emergency lane. | IT and change board | Signed-off process. |
| **This quarter** | Plan to retire or isolate every legacy system. | IT and business owners | Dated plan per system. |
| **This quarter** | Report patch latency to the board monthly. | CIO or IT director | First report delivered. |

PITFALLS

# Six ways the gap stays open

Each is common and each is avoidable.

**01** **Waiting for the monthly window**

Exploited flaws are handled in the same slot as routine updates.

**02** **Trusting the severity score alone**

A medium flaw being exploited on a public server matters more than a critical one nobody can reach.

**03** **Forgetting the hidden copies**

Test, disaster recovery and vendor-supplied systems are missed.

**04** **Treating mitigation as a fix**

Firewall rules stay in place for months and the patch never happens.

**05** **Patching without hunting**

Systems are fixed after attackers are already inside.

**06** **Measuring nothing**

Nobody can say how long fixes take, so nothing improves.

ASK NOW

# Questions for your team and your MSP

Use these at your next service review.

### YOUR OWN TEAM

- ? Do we know every internet-facing system we run, and who owns it?
- ? How long did PHP2Shell and the October Windows zero-day take us to fix?
- ? Which systems run software that no longer gets security fixes?

### YOUR MANAGED SERVICE PROVIDER

- ? Will you alert us within hours when a flaw we run is being exploited?
- ? Can you patch outside the monthly cycle, and what is your target time?
- ? Do you check for compromise after an emergency patch, and report it?

CHECKLIST

# Is your patch gap closing?

Tick what is in place today. Anything blank is your next step.

- [ ] **A live inventory with an owner for every system**
  Including internet-facing and supplier-managed systems.

- [ ] **Daily watch on exploited-flaw alerts**
  CISA's exploited list, NCSC alerts and vendor advisories.

- [ ] **Patch targets agreed by risk tier**
  Hours to days for exploited and exposed flaws.

- [ ] **A pre-approved emergency patch lane**
  No meeting needed to act.

- [ ] **Testing in rings with rollback ready**
  Hours for urgent fixes, days for routine ones.

- [ ] **A plan for every legacy system**
  Upgrade, replace, isolate or formally accept.

- [ ] **Patch latency reported monthly**
  To the leadership team or board.

GLOSSARY

# Words worth knowing

**Patch latency**

The time between a vendor releasing a fix and it being installed on your systems.

**CVE**

A unique ID for a publicly known security flaw.

**CVSS**

A 0 to 10 severity score. It does not tell you whether a flaw is being exploited.

**Known exploited vulnerability**

A flaw with evidence of real attacks. CISA keeps a public list.

**Zero-day**

A flaw exploited before a fix exists.

**Emergency patch**

A fix released or deployed outside the normal schedule because of urgent risk.

**Patch rings**

Deploying updates in waves, starting with a small pilot group.

**Hotpatching**

Applying security updates without a restart.

**Compensating control**

A temporary measure that reduces risk until a fix is installed.

**End of life**

Software that no longer receives security fixes.

**Attack surface**

Everything an attacker could reach, especially from the internet.

**MSP**

Managed service provider. A partner that runs some or all of your IT.

TRANSPUTEC

# Close the gap before attackers do.

Transputec helps UK organisations build a live view of their estate, patch exploited flaws in days and keep everything else on a steady, tested rhythm.

Book a patch gap review

transputec.com

READ THE COMPANION REPORT

The Patch Gap · Companion intelligence report

